



Circana AI Pilot: Unified Architecture Design & Implementation Plan

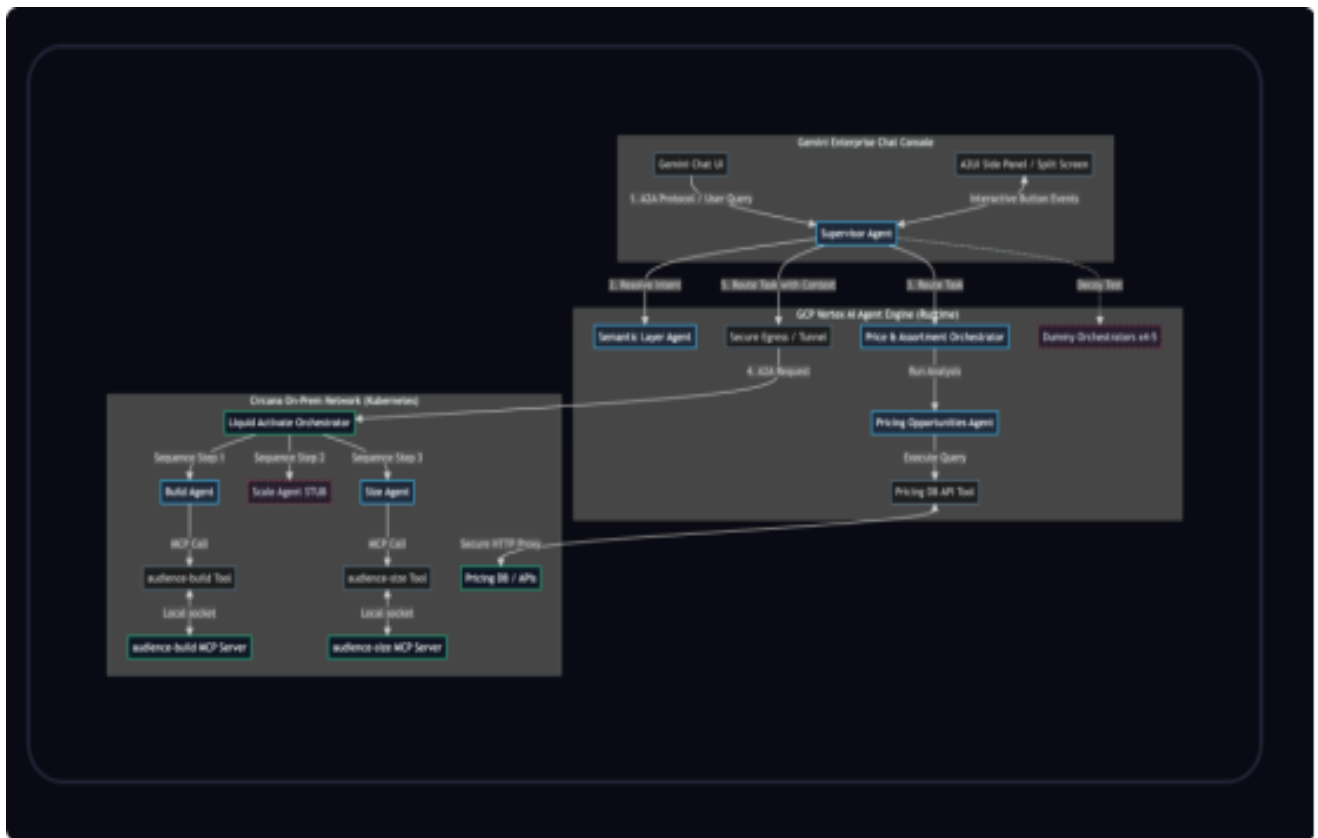
This master document consolidates all architectural designs, operational sequence flows, network connectivity structures, SSO identity federation details, frontend rendering constraints, and staging directory plans for the Circana AI Pilot.

Table of Contents

1.  1. Multi-Agent Hierarchy & Topology
2.  2. Architectural Dimensions (Numbers & Relationships)
3.  3. Hybrid Deployment & Agent-to-Agent (A2A) Execution
4.  4. Google Cloud Platform Infrastructure Mapping
5.  5. Operational Sequence Flow
6.  6. Hybrid Network Connectivity Architecture
7.  7. Federated Identity & SSO Architecture
8.  8. A2UI Frontend Rendering Analysis
9.  9. Staging Directory Layout & Packaging Build Configuration
10.  10. No-Code Flow Agent vs. Code-First ADK Analysis
11.  11. Open Questions & Decisive Pivot Points

1. Multi-Agent Hierarchy & Topology

To meet Circana's requirement of semantic reasoning, routing precision (decoy agents), and multi-step orchestration, we adopt a **3-Tier Hierarchical Multi-Agent System** using the Google GenAI **Agent Development Kit (ADK)**.



2. Architectural Dimensions (Numbers & Relationships)

A. Agent Count

1 Coordinator Agent: SupervisorAgent (Root Agent)

2 Real Solution Orchestrators:

PricingAssortmentOrchestrator

LiquidActivateOrchestrator

4 Real Specialist Agents:

SemanticLayerAgent (Shared entity resolver)

PricingOpportunitiesAgent (Query builder/executor)

AudienceBuildAgent (Builds segments)

AudienceSizeAgent (Calculates population numbers)

1 Stubbed Specialist Agent:

AudienceScaleAgent (Mocked look-alike finder)

4-5 Decoy Orchestrators:

Registered with valid descriptions (e.g. LoyaltyCampaignOrchestrator ,

MarkdownOptimizationOrchestrator , etc.) but containing no active sub-agents or real tools. Used to verify

the Supervisor's routing precision (if a decoy is invoked, it counts as a routing failure). **Total Agents: ~12-13**

agents (7 real, 1 stub, 4-5 decoys).

B. Tool and MCP Relationships

2 On-Premises MCP Servers:

audience-build server: Hosts the audience-build tool (parameters: product, time range, spend criteria).

audience-size server: Hosts the audience-size tool (parameters: audience ID, partner options). **1**

Native API Tool:

pricing-opportunities tool: Standard Python FunctionTool invoking Circana's pricing metrics API. **Total**

Tools: 3 core tools exposed to specialist agents.

C. Network Hops & Interaction Flow (IP Hops)

1. **User → Gemini UI:** User inputs query.
2. **Gemini UI → Vertex Agent Engine (A2A):** Gemini Enterprise forwards the query to the Supervisor agent's container.
3. **Supervisor Agent → Semantic Layer Agent (Intra-Container):** Supervisor resolves domain entity definitions (e.g., mapping "attrition" to "lost households").
4. **Supervisor Agent → Pricing & Assortment Orchestrator (Intra-Container):** Supervisor requests the pricing analysis.
5. **Pricing Opportunities Agent → Circana Pricing API (Egress Call):** Egresses via Secure HTTP Proxy or Private Service Connect (PSC) to retrieve the table of attrition products.
6. **Supervisor Agent → User (A2UI Inline Rendering):** Returns the Table A2UI component as a **Human-In-The-Loop (HITL) Checkpoint**. The user validates/selects the top product.
7. **User → Supervisor Agent (A2A):** User confirms selection.
8. **Supervisor Agent → Liquid Activate Orchestrator (Intra-Container):** Initiates audience workflow.
9. **Audience Build Agent → audience-build MCP Server (Egress Call):** Calls the on-premise MCP server to construct the audience segment.
10. **Audience Size Agent → audience-size MCP Server (Egress Call):** Calls the on-premise MCP server to estimate size.
11. **Supervisor Agent → Gemini UI (A2UI Sidebar Rendering):** Returns the final interactive Audience Dashboard (KPIs, Charts, and Activation buttons).

3. Hybrid Deployment & Agent-to-Agent (A2A) Execution

To satisfy Circana's target environment constraints, the multi-agent system runs across a hybrid infrastructure split between Google Cloud Platform and Circana's on-premises network:

A. Runtime Distribution

GCP Cloud-Native Runtime (Vertex AI Agent Engine):

Agents: SupervisorAgent (Root), PricingAssortmentOrchestrator , DecoyOrchestrators . **Rationale:** Needs low-latency access to the Gemini Enterprise A2A webhooks, the A2UI catalog renderers, and Google Cloud's safety/observability suites (Model Armor, Memory Bank).

On-Premises Container Runtime (Circana Kubernetes Cluster):

Agents: LiquidActivateOrchestrator , AudienceBuildAgent , AudienceSizeAgent .

Rationale: These agents must execute queries directly against Circana's proprietary audience databases and science models to guarantee that raw customer data never leaves the corporate network.

B. Agent-to-Agent (A2A) Communication Bridge

The GCP SupervisorAgent communicates with the on-premises LiquidActivateOrchestrator over the private PSC tunnel using standard **A2A HTTP/gRPC API requests**.

An on-premises API gateway acts as the secure entry point, routing calls directly to the on-premises agent pods.

C. Federated Token Propagation

When calling the on-premises agent, the GCP Supervisor forwards the user's Microsoft Entra ID token in the request headers.

The on-premises agent validates this token locally to enforce user-level data permissions, ensuring complete identity synchronization across the cloud-premises boundary.



4. Google Cloud Platform Infrastructure Mapping

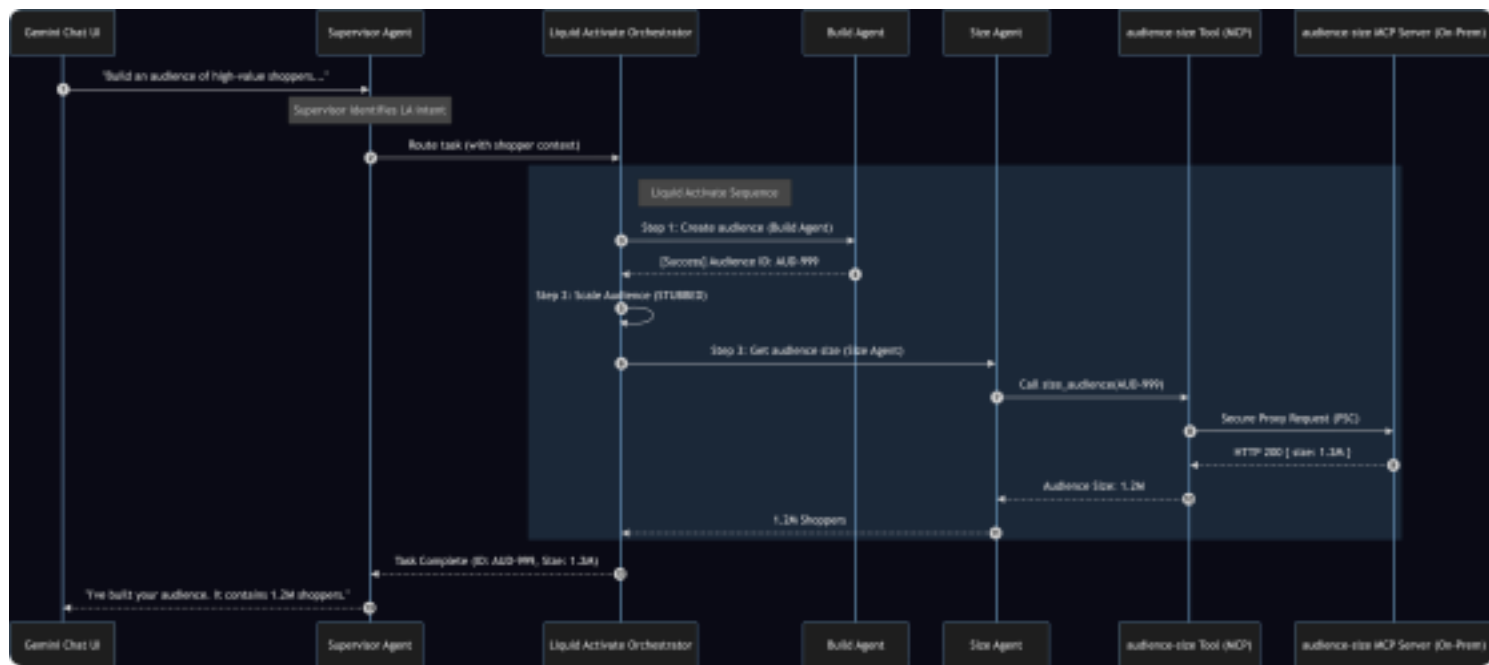
Component	Platform Infrastructure	Description
Chat Entry Point	Gemini Enterprise App	The primary user chat console. Supports SSO (Microsoft Entra), custom agent selection ("From your organization" tab), and split-screen layouts.
Agent Runtime	Vertex AI Agent Engine	Serverless Python container runtime. Hosts the ADK Supervisor and sub agents, handling session context and execution.
A2A / A2UI Wrapper	a2a-sdk Python Web Server	Receives tasks, converts GenAI outputs (wrapped in <a2ui-json>) into rich component payloads, and forwards interactive UI events.
Short-Term Memory	ADK Session Service	In-memory or Redis-backed session state capturing active user context across task-chaining steps.

Long-Term Memory	Agent Platform Memory Bank (Primary) & Vector DB / Spanner	Native long-term memory repository (performing automatic LLM-based extraction and consolidation across sessions) combined with Spanner for domain ontology/grounding.
Private Connectivity	Private Service Connect (PSC)	Secures communication between the GCP Vertex AI Agent Engine and the on-premise Circana network. A transit VM can act as an outbound reverse proxy.
Safety Shield	Model Armor	Intercepts user prompts and agent outputs to redact PII, restrict sensitive information leakage, and enforce safety guidelines.
Logging & Cost Control	Cloud Logging & BigQuery	Aggregates session telemetry, latency metrics, and API call costs per step for auditing.



5. Operational Sequence Flow

A. Sequence Diagram



B. Execution Details

Phase A: Pricing Assortment & HITL Product Selection

- User Query:** The user requests: "Across my portfolio, identify products where price increases over the past 52 weeks drove buyer attrition; products that lost households, not just volume..."
- Terms Resolution:** SupervisorAgent asks SemanticLayerAgent to map "buyer attrition" to lost_households .
- Data Acquisition:** PricingOpportunitiesAgent queries Circana's pricing database on-premise.
- HITL Split-Screen:** An interactive table of attrition products is rendered to the user in a split-screen panel,

pausing the pipeline.

Phase B: Liquid Activate & Sizing Pipeline

5. **Select Callback:** The user selects a product. The Supervisor invokes the LiquidActivateOrchestrator . 6.

Audience Creation: AudienceBuildAgent asks the on-premises audience-build MCP server to materialize the cohort.

7. **Audience Sizing:** AudienceSizeAgent asks the on-premises audience-size MCP server to estimate the size.

8. **Dashboard Presentation:** Sizing counts are shown to the user on A2UI KPI cards.

Phase C: Activation & Export

9. **Channel Activation:** The user selects partners (LiveRamp, Google) and triggers the export tool to complete the workflow.

6. Hybrid Network Connectivity Architecture

A. Connectivity Map



B. Connectivity Options

Option A: Transit VM Proxy (Recommended for Pilot)

Deploy a lightweight VM in the GCP VPC. The agent container connects to the Peered VPC and routes tool requests to the VM's internal IP proxy. The VM forwards queries over IPsec VPN to the on-prem endpoints. *Pros:* Quick 3-day turnaround.

Option B: Private Service Connect (PSC)

Map on-premise MCP services directly to a GCP PSC endpoint.

Pros: Enterprise-grade structure, no proxy middleman.

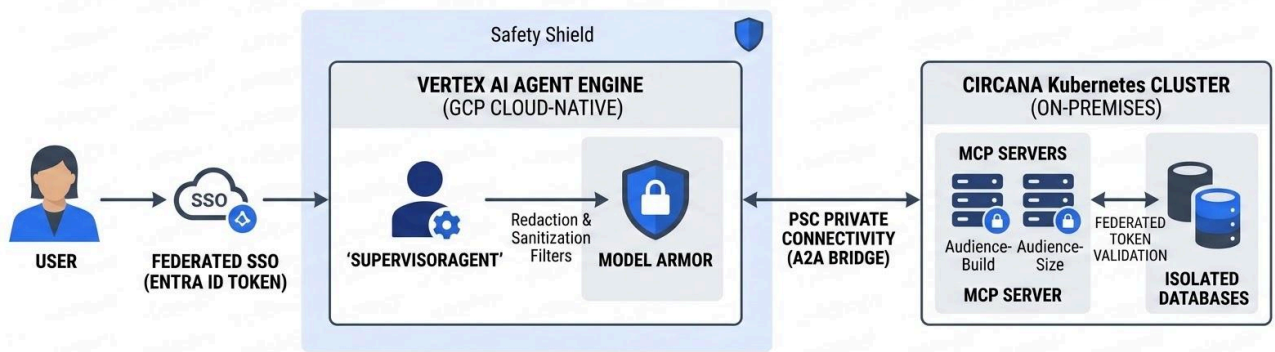
Option C: Simulation / Local Mocking (Backup)

Mock tools inside the agent container.

Pros: Bypasses network setups completely to guarantee pilot demonstration.

C. Data Protection (Model Armor)

Layered Defense & Hybrid Data Protection Flow

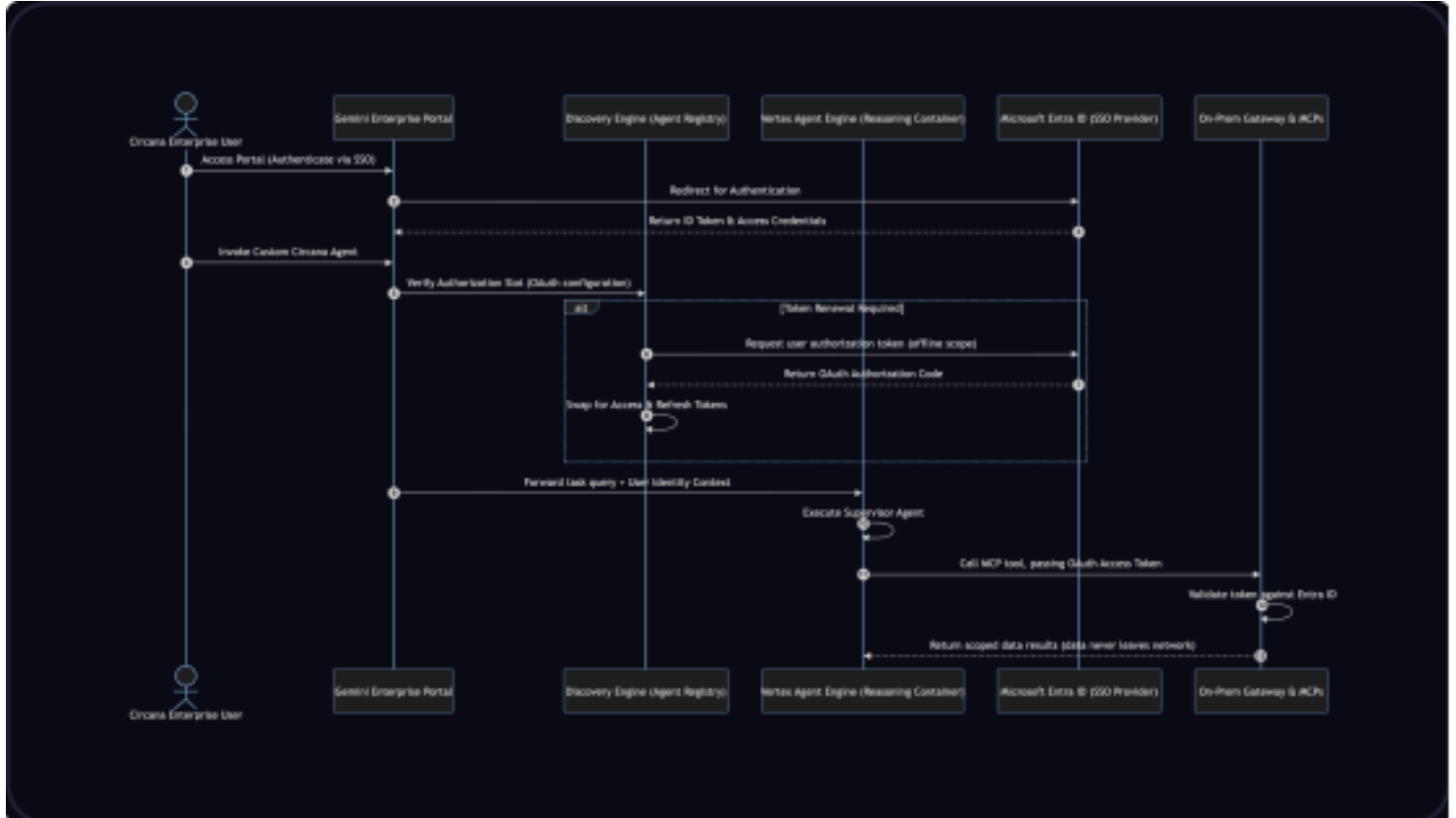


Redaction Filters: Prevents PII leaks from User Input to the Supervisor.

Sanitization Filters: Scrubs sensitive database schemas from agent outputs back to the chat UI.

🔑 7. Federated Identity & SSO Architecture

A. Identity delegation Flow



B. Platform Authentication Mapping

User Sign-On: OIDC Federation via Microsoft Entra ID.

Agent-to-Agent (A2A): Google IAM default bearer tokens.

Agent-to-Tool: Implements OAuth impersonation using credentials fetched from the Discovery Engine registry slots.

C. Registry Slot Configuration

Discovery Engine slots cache tokens and authorization configs securely:

```
{  
  "name": "projects/<PROJECT_ID>/locations/global/authorizations/combined-auth-v55",  
  "serverSideOAuth2": {  
    "clientId": "<ENTRA_CLIENT_ID>",  
    "clientSecret": "<ENTRA_CLIENT_SECRET>",  
    "authorizationUri": "https://login.microsoftonline.com/<TENANT_ID>/oauth2/v2.0/authorize?client_id=<CLIENT_ID>&response_type=code",  
    "tokenUri": "https://login.microsoftonline.com/<TENANT_ID>/oauth2/v2.0/token" }  
}
```

Redirect URI: Must point to <https://vertexaisearch.cloud.google.com/static/oauth/oauth.html> . **Offline Access:** Scope MUST include `offline_access` to cache **Refresh Tokens** for background tasks.



8. A2UI Frontend Rendering Analysis

A. Core Component Channels

Native Declarative JSON: Schema outputs (like tables, Vega Charts) are rendered natively by the host console (built with Lit/Angular by Google).

Sandboxed Widgets (`WebFrameSrcdoc`): Custom HTML documents output by the agent stream as raw strings inside `<a2ui-json>` elements.

B. Sandbox Framework Constraints (React / Angular / Lit)

CSP (`connect-src 'none'`): Inside the widget iframe, no outgoing network queries (`fetch()` , Websockets) are permitted. All assets must be inline.

Trusted Types: Modifying DOM using standard framework compilers (like React .innerHTML modifications) violates strict platform sanitization rules.

Latency: Large framework bundle sizes bloat the streaming context.

Recommendation: **Vanilla HTML5, CSS3, and JavaScript** communicating with the parent frame using `postMessage` listeners is the supported, secure pattern.



9. Staging Directory Layout & Packaging Build Configuration

```

Circana_POC/
├── .env # Local development credentials
├── pyproject.toml # Python packaging configuration
├── requirements.txt # Declared container dependencies
├── deploy.py # Script to package and deploy Reasoning Engine
├── register.py # Script to register agent card in Gemini Enterprise
├── agent.py # Root Bridge: imports agents.circana_pilot_agent.agent
├── executor.py # Root Bridge: imports agents.circana_pilot_agent.executor
├── agents/
│   └── circana_pilot_agent/
│       ├── __init__.py # Module indicator
│       ├── agent.py # Core ADK agent tree definitions
│       ├── executor.py # Core A2A task executor class
│       ├── tools.py # Function tool definitions (MCP clients)
│       └── components.py # Premium A2UI layout builder and HTML templates

```

B. Staging Bridge Patterns

To allow the local ADK emulator CLI to run without path import exceptions, we use bridge files at the root:

```

# Root Bridge: agent.py
try:
    from agents.circana_pilot_agent.agent import root_agent
except ImportError:
    from agent import root_agent

```



10. No-Code Flow Agent vs. Code-First ADK Analysis

A. Flow Agent / Dialogflow CX

Pros: Visual flow builder, easy visual slot filling.

Cons: Returning nested custom `<a2ui-json>` components is extremely complex, decoy routes cannot be programmatically validated, and mounting raw MCP clients is not natively integrated.

B. Python ADK (Recommended)

Pros: Native support for `A2uiSchemaManager`, precise routing configuration for decoy testing, and easy async

MCP client wrappers.

Cons: Requires staging packaging templates.

Recommendation: Utilize the **Code-First Python ADK** approach on Vertex AI Agent Engine.

? 11. Open Questions & Decisive Pivot Points

1. **Private Access for the Pilot:** Connect via a transit VM in the next 3 days, or **mock/simulate** tools to guarantee pilot demo schedules?
2. **Entra Identity Federation:** Defer Microsoft Entra federation to post-pilot using service accounts, or implement full delegation now?
3. **Decoy Naming Conventions:** Use specific real-world mock titles, or generate standard retail mock